MODULE 25 · SECTION 3 - STORING THE DATA

Lakes and Lakehouses

Lake zones in Cloud Storage, file formats, Lakehouse tables with metadata caching and fine-grained security, Iceberg, object tables and ObjectRef, monitoring the lake.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 25 of 45

admin@sthithapragna.com

A lake is just files until something governs them. **Lakehouse tables give files the rules of a warehouse.**

Load it, or leave it in the lake?

LOAD INTO BIGQUERY

Native tables

- Fastest queries and every feature
- Streaming, DML, time travel
- BigQuery manages the storage
- Other engines read through APIs

LEAVE IN THE LAKE

Open files

- One copy shared by Spark and BigQuery
- Open formats, no lock-in
- You manage files and layout
- Lakehouse tables add governance

THE TIEBREAKER

Only BigQuery uses it: load it. Spark and BigQuery share it: keep one open copy in the lake.

Larkspur's lake, zone by zone

01

Landing

Files as delivered. Short-lived, soft delete off.

02

Raw

Immutable history. Retention, colder classes.

03

Curated

Clean Parquet or Iceberg. Partitioned, compacted.

04

Consumption

Lakehouse tables and marts that people query.

THE PATTERN

One bucket per zone, so permissions, lifecycle and retention are set zone by zone.

Formats, file sizes and folders

- **Parquet** or **ORC** for analysis: columnar, compressed, read only the columns asked for.
- **Avro** for loading and exchange. CSV and JSON belong in landing only.
- Millions of **small files** slow everything. Compact them into larger files.
- Hive layout: **key=value** folders, such as run_date=2026-10-09, in a fixed order.

FOR LARKSPUR

Google notes that listing millions of files can take several minutes. Larkspur's history had 2.9 million.

External table or Lakehouse table?

STANDARD EXTERNAL TABLE

For a quick look

- Users need access to the bucket too
- No row or column security
- Lists files at query time
- Fine for ad hoc discovery

LAKEHOUSE TABLE

For governed access

- A connection's service account reads the files
- Users need access to the table only
- Row, column and masking policies
- Metadata caching speeds planning

THE TIEBREAKER

Governance needed: a Lakehouse table. A quick ad hoc look with no governance: an external table.

Settlement runs, as a Lakehouse table

```
1  CREATE EXTERNAL TABLE curated_lake.settlement_runs
2  WITH PARTITION COLUMNS (run_date DATE)
3  WITH CONNECTION `lk-warehouse-prod.europe-west2.lk-lake-conn`
4  OPTIONS (
5    format = 'PARQUET',
6    uris = ['gs://lk-lake-prod/settlement/*'],
7    hive_partition_uri_prefix = 'gs://lk-lake-prod/settlement',
8    require_hive_partition_filter = TRUE,
9    max_staleness = INTERVAL 4 HOUR, metadata_cache_mode = 'MANUAL');
10
11 CALL BQ.REFRESH_EXTERNAL_METADATA_CACHE('curated_lake.settlement_runs');
```

3 The connection makes this a Lakehouse table. Its service account reads the files.

9-11 Cache the file list, and refresh it as soon as the nightly job finishes.

FOR LARKSPUR

The Spark job finishes by 02:00, so a manual refresh fits better than a timer.

Caching what the lake looks like

- Caches file names, partitions and row counts, plus **statistics** for Parquet.
- Staleness from **30 minutes to 7 days**. Older than that, queries list files again.
- **Automatic** refresh every 30 to 60 minutes, or **manual** on your schedule.
- Refreshes use query capacity. Give them a **BACKGROUND** reservation.

> THE TIEBREAKER
>
> Files land at a known time: manual, after the job. Files land at random: automatic.

Row and column rules on files

- **Row access policies** and **policy tags** work on Lakehouse tables.
- **Dynamic masking** works on Lakehouse tables over Cloud Storage.
- Spark, Trino and Presto read through the **Storage Read API**, which enforces them.
- Anyone with direct **bucket access** can still read every file.

> THE TRAP
>
> Granting analysts read access on the bucket lets them walk round every policy. Grant the table only.

Iceberg in the lakehouse, briefly

- **Iceberg managed tables**: BigQuery writes, with DML and streaming, into your bucket.
- Spark-written Iceberg registers in the **Lakehouse runtime catalog**.
- Iceberg tracks files in its own metadata, so no folder listing is needed.
- Module 09 covered who writes which kind. The rule still holds.

> REMEMBER
>
> Needs updates and streaming on one open copy: an Iceberg managed table.

Object tables or ObjectRef?

OBJECT TABLE

A table of files

- One row per object: uri, type, size
- Read-only, through a connection
- Feeds AI functions and Document AI
- Row policies decide which files

OBJECTREF

A file inside a row

- A STRUCT column in a normal table
- Links a record to its file
- Mixes structured and unstructured
- Build it from an object table's ref

FOR LARKSPUR

The eighteen thousand pages of manuals become an object table, ready for RAG in module 31.

Monitoring and cost

- Find cache refresh jobs in **INFORMATION_SCHEMA.JOBS**, and check cache use per query.
- **Storage Insights datasets** put bucket and object metadata into BigQuery, daily.
- On-demand queries over files bill **bytes read**. Parquet and partition filters cut it.
- **Lifecycle rules** per zone keep landing clean and raw cold.

> HONEST LIMIT
>
> Storage Insights needs a Storage Intelligence subscription. Check before you rely on it.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Spark and BigQuery share **one copy**	Open files in the lake, Lakehouse tables
Row security on files, **no bucket access**	Lakehouse table with a connection
Queries spend minutes **listing files**	Metadata caching, and compaction
Files land at a **known time**	Manual cache refresh after the job
Analyse **PDFs or images** from SQL	Object table and AI functions
Updates and streaming on open files	Iceberg managed table

THE PATTERN

Lake questions turn on three words: share, govern, or speed up.

> Westgrid analysts query Larkspur's settlement Parquet files through a standard external table, and hold read access on the bucket.
>
> They must now see only rows for their own grid region, without a second copy of the data.

What should the team do?

A Keep the external table, and give Westgrid a view filtered to its region

B Copy Westgrid's rows into a separate bucket for them every night

C Put the bucket inside a VPC Service Controls perimeter

D A Lakehouse table with a row access policy, and remove bucket access

◆ PAUSE AND DECIDE

What should the team do?

A A filtered view

Bucket access still lets them read every file directly.

B A nightly copy

A second copy, which the brief rules out.

C A perimeter

Controls where requests come from, not which rows are seen.

D Lakehouse table plus policy

The connection reads files. Rows are filtered per user.

THE SIGNAL

Row-level rules on files, and no second copy: a Lakehouse table, with bucket access removed.

> A Lakehouse table covers two million Parquet files. Queries spend minutes before reading any data.
>
> A Spark job rewrites the files once a night, finishing by 02:00, and the files do not change during the day.

What should the team do?

A. Enable automatic metadata caching, with a 30-minute staleness

B. Convert the files to CSV, which BigQuery can list more quickly

C. Enable manual metadata caching, refreshed right after the job

D. Buy more slots, so that listing the files finishes faster

◆ PAUSE AND DECIDE

What should the team do?

A Automatic, 30 minutes

Refreshes all day for files that never change, and can still go stale.

B CSV

Larger files, no column pruning, and just as many to list.

C Manual, after the job

One refresh a night, exactly when the files change.

D More slots

Listing files is not the part that more slots fix.

THE SIGNAL

Minutes spent listing, files that change at a known time: manual metadata caching.

> Larkspur keeps 18,000 pages of engineering manuals as PDFs in gs://lk-manuals.
>
> Data scientists want to extract the text with Document AI from SQL, without copying the files anywhere.

What should the team set up?

A. Load each PDF into a BYTES column of a standard BigQuery table

B. A Dataflow job that copies each PDF into a Bigtable row

C. An external table over the bucket, with its format set to PDF

D. An object table over the bucket, then ML.PROCESS_DOCUMENT

◆ PAUSE AND DECIDE

What should the team set up?

A BYTES column

Copies every file, which the brief rules out.

B Bigtable

Another copy, in a store with no SQL route to Document AI.

C External table as PDF

PDF is not an external table format.

D Object table

Files stay put. SQL sends them to Document AI.

THE SIGNAL

Unstructured files, analysed from SQL, without copying: an object table.

> The settlement team's Spark jobs and the BigQuery analysts both use the curated settlement data.
>
> It must exist once, in an open format, and BigQuery must be able to update rows with DML and stream new rows in.

Which storage choice fits best?

A Native BigQuery tables, with Parquet copies exported for Spark

B Standard external tables over Parquet files that Spark writes

C An Iceberg managed table, written by BigQuery, read by Spark

D Keep the Hive tables on Hadoop, and query them through Omni

◆ PAUSE AND DECIDE

Which storage choice fits best?

A Native plus exports

Two copies, and the exports are always behind.

B External tables

Read-only from BigQuery: no DML, no streaming.

C Iceberg managed table

One open copy, with DML and streaming from BigQuery.

D Hadoop and Omni

Omni reaches other clouds, not a closing data centre.

THE SIGNAL

One open copy that BigQuery updates and streams into: an Iceberg managed table.

Module 25 in five lines

- Zones in **separate buckets**: landing, raw, curated, consumption.
- **Parquet** in Hive folders, and compact the small files.
- **Lakehouse tables** read through a connection, with row and column rules.
- **Metadata caching**: manual after a known job, automatic otherwise.
- **Object tables** for files, **Iceberg managed tables** for updates.

COMING UP

Next: module 26, Knowledge Catalog and federated governance.